University of Wah

Department of Computer Science

Submit to:

Ms. Hassan Shafaqat

Submit by:

Muhammad Talha

Course:

Artificial Intelligence Lab_202L

Reg No:

UW-24-CS-BS-101

Section:

BS-CS-3rd-B

Question # 01

```python
import random

W, H = 10, 10
START = (0, 0)
GOAL = (9, 9)

cost = [[1 for _ in range(W)] for _ in range(H)]
blocked = set()

def h(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def neighbors(x, y):
    for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
        nx, ny = x+dx, y+dy
        if 0 <= nx < W and 0 <= ny < H:
            yield nx, ny

def dynamic_astar():
    open_list = [(h(START, GOAL), START)]
    g = {START: 0}
    parent = {}
    log = []

    for step in range(100):
        if not open_list:
            break

        for nx, ny in neighbors(*current):
            if (nx, ny) in blocked:
                continue

            new_g = g[current] + cost[ny][nx]
            if (nx, ny) not in g or new_g < g[(nx, ny)]:
                g[(nx, ny)] = new_g
                parent[(nx, ny)] = current
                open_list.append((new_g + h((nx, ny), GOAL), (nx, ny)))

        rx, ry = random.randint(0, W-1), random.randint(0, H-1)
        if random.random() < 0.3:
            blocked.add((rx, ry))
            log.append(f"Blocked {(rx, ry)}")
        else:
            cost[ry][rx] = random.randint(1, 5)
            log.append(f"Cost change {(rx, ry)}")
```

```
    path = []
    node = GOAL
    while node in parent:
        path.append(node)
        node = parent[node]
    path.append(START)
    path.reverse()

    return path, log

path, log = dynamic_astar()

print("Path:", path)
print("\nLast 10 log entries:")
for l in log[-10:]:
    print(l)
```

Output

```
Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4), (4, 5), (4, 6), (4, 7), (4, 8), (5, 8), (5, 9), (6, 9), (7, 9), (8, 9), (9, 9)]

Last 10 log entries:
Cost change (6, 4)
Cost change (7, 4)
Blocked (2, 1)
Blocked (4, 7)
Cost change (6, 5)
Blocked (1, 5)
Blocked (3, 6)
Blocked (1, 0)
Cost change (5, 2)
Cost change (5, 9)
```

Question # 03

```
import random

class Game:
    def __init__(self, size, win):
        self.n = size
        self.k = win
        self.board = [[' ']*size for _ in range(size)]
        self.turn = 'X'

    def moves(self):
        return [(i,j) for i in range(self.n)
                      for j in range(self.n)
                      if self.board[i][j] == ' ']

    def play(self, move):
        i,j = move
        self.board[i][j] = self.turn
        self.turn = 'O' if self.turn == 'X' else 'X'

    def win_check(self):
        for i in range(self.n):
            for j in range(self.n):
                if j+self.k <= self.n:
                    line = [self.board[i][j+x] for x in range(self.k)]
                    if len(set(line)) == 1 and line[0] != ' ':
                        return line[0]
                if i+self.k <= self.n:
                    line = [self.board[i+x][j] for x in range(self.k)]
                    if len(set(line)) == 1 and line[0] != ' ':
                        return line[0]
        return None
```

```
    def terminal(self):
        return self.win_check() is not None or not self.moves()

def heuristic(game, player):
    score = 0
    for row in game.board:
        score += row.count(player)
    return score

class EasyAI:
    def __init__(self, symbol):
        self.symbol = symbol

    def choose(self, game):
        best = None
        best_score = -999

        for m in game.moves():
            temp = Game(game.n, game.k)
            temp.board = [r[:] for r in game.board]
            temp.turn = game.turn
            temp.play(m)

            score = heuristic(temp, self.symbol)

              if random.random() < 0.2:
                  score -= 2

              if score > best_score:
                  best_score = score
                  best = m

          return best if best else random.choice(game.moves())

def run():
    size = random.choice([3,4,5])
    win = random.choice([3,4])
    game = Game(size, win)
    ai = EasyAI('X')

    while not game.terminal():
        if game.turn == 'X':
            move = ai.choose(game)
        else:
            move = random.choice(game.moves())

        game.play(move)

    print("Board:", size, " Win:", win)
    for r in game.board:
        print(r)
    print("Winner:", game.win_check())

run()
```

Output

```
Board: 5  Win: 4
['X', 'X', 'X', 'X', ' ']
[' ', ' ', 'O', ' ', ' ']
['O', ' ', 'O', ' ', ' ']
[' ', ' ', ' ', ' ', ' ']
[' ', ' ', ' ', ' ', ' ']
Winner: X
```

Question # 04

```python
import random

resources = {
    "CPU": 10,
    "RAM": 10,
    "NET": 10
}

services = {
    "A": {"CPU": 4, "RAM": 3, "NET": 2},
    "B": {"CPU": 5, "RAM": 4, "NET": 4},
    "C": {"CPU": 3, "RAM": 4, "NET": 3}
}

allocation = {
    s: {r: random.randint(0, services[s][r]) for r in resources}
    for s in services
}


def penalty(allocation):
    total_use = {r: 0 for r in resources}

    for s in allocation:
        for r in resources:
            total_use[r] += allocation[s][r]

    p = 0
    for r in resources:
        if total_use[r] > resources[r]:
            p += total_use[r] - resources[r]

    return p


def improve(allocation):
    best = allocation
    best_penalty = penalty(allocation)
```

```
    for s in services:
        for r in resources:
            new_alloc = {k: v.copy() for k, v in allocation.items()}
            change = random.choice([-1, 1])

            new_value = new_alloc[s][r] + change
            if 0 <= new_value <= services[s][r]:
                new_alloc[s][r] = new_value
                p = penalty(new_alloc)

                if p <= best_penalty:
                    best = new_alloc
                    best_penalty = p

    return best, best_penalty

best_solution = allocation
best_score = penalty(allocation)
```

```
best_solution = allocation
best_score = penalty(allocation)

for step in range(100):
    allocation, score = improve(allocation)

    if score <= best_score:
        best_solution = allocation
        best_score = score

    if step % 20 == 0:
        resources["CPU"] = random.randint(7, 12)

print("Final Allocation:")
for s in best_solution:
    print(s, best_solution[s])

print("Final Penalty:", best_score)
```

Output

```
Final Allocation:
A {'CPU': 2, 'RAM': 3, 'NET': 1}
B {'CPU': 5, 'RAM': 3, 'NET': 0}
C {'CPU': 0, 'RAM': 0, 'NET': 0}
Final Penalty: 0
```

Question # 05

```python
import re
import random

def tokenize(text):
    text = text.lower()
    return re.findall(r"[a-zA-Z']+", text)

INTENTS = {
    "urgent": ["now", "asap", "immediately", "urgent", "fast"],
    "frustrated": ["bad", "worst", "angry", "annoyed", "problem", "not", "working"],
    "satisfied": ["thanks", "great", "good", "awesome", "love"],
    "question": ["how", "what", "why", "when"]
}

def score_intents(tokens):
    scores = {i: 0 for i in INTENTS}
    for t in tokens:
        for intent in INTENTS:
            if t in INTENTS[intent]:
                scores[intent] += 1
    return scores

def uncertainty(scores):
    vals = sorted(scores.values(), reverse=True)
    if len(vals) < 2:
        return 1.0
    return abs(vals[0] - vals[1]) / (vals[0] + 1)
def reanalyze(scores, u):
    if u < 0.4:
        for k in scores:
            scores[k] += random.choice([0, 1])
    return scores

def classify(text):
    trace = []

    tokens = tokenize(text)
    trace.append("Tokens: " + str(tokens))

    scores = score_intents(tokens)
    trace.append("Initial Scores: " + str(scores))

    u = uncertainty(scores)
    trace.append("Uncertainty: " + str(round(u, 2)))

    scores = reanalyze(scores, u)
    trace.append("Reanalyzed Scores: " + str(scores))

    intent = max(scores, key=scores.get)

    return intent, round(1 - u, 2), trace
```

```python
inputs = [
    "This service is GREAT",
    "why this not working asap",
    "ok fine whatever",
    "hello need help now"
]

for text in inputs:
    intent, confidence, trace = classify(text)
    print("\nInput:", text)
    print("Predicted Intent:", intent)
    print("Confidence:", confidence)
    print("Reasoning Trace:")
    for t in trace:
        print(" -", t)
```

Output

```
Input: This service is GREAT
Predicted Intent: satisfied
Confidence: 0.5
Reasoning Trace:
 - Tokens: ['this', 'service', 'is', 'great']
 - Initial Scores: {'urgent': 0, 'frustrated': 0, 'satisfied': 1, 'question': 0}
 - Uncertainty: 0.5
 - Reanalyzed Scores: {'urgent': 0, 'frustrated': 0, 'satisfied': 1, 'question': 0}

Input: why this not working asap
Predicted Intent: frustrated
Confidence: 0.67
Reasoning Trace:
 - Tokens: ['why', 'this', 'not', 'working', 'asap']
 - Initial Scores: {'urgent': 1, 'frustrated': 2, 'satisfied': 0, 'question': 1}
 - Uncertainty: 0.33
 - Reanalyzed Scores: {'urgent': 1, 'frustrated': 2, 'satisfied': 0, 'question': 1}

Input: ok fine whatever
Predicted Intent: frustrated
Confidence: 1.0
Reasoning Trace:
 - Tokens: ['ok', 'fine', 'whatever']
 - Initial Scores: {'urgent': 0, 'frustrated': 0, 'satisfied': 0, 'question': 0}
 - Uncertainty: 0.0
 - Reanalyzed Scores: {'urgent': 0, 'frustrated': 1, 'satisfied': 1, 'question': 0}

Input: hello need help now
Predicted Intent: urgent
Confidence: 0.5
Reasoning Trace:
 - Tokens: ['hello', 'need', 'help', 'now']
 - Initial Scores: {'urgent': 1, 'frustrated': 0, 'satisfied': 0, 'question': 0}
 - Uncertainty: 0.5
 - Reanalyzed Scores: {'urgent': 1, 'frustrated': 0, 'satisfied': 0, 'question': 0}
```